

ΕΡΓΑΣΤΗΡΙΟ ΜΕΤΑΓΛΩΤΤΙΣΤΩΝ

ΜΕΡΟΣ Α-2

Κωδικοποίηση αυτομάτων πεπερασμένων καταστάσεων μέσω FSM



Τμήμα ΜΕΤ - 04

Ομάδα 3

ΛΕΟΝΤΑΡΙΔΗΣ ΔΗΜΗΤΡΙΟΣ - ΑΜ: 171029

ΝΤΑΛΙΠΑΪ ΒΕΣΕΛ - ΑΜ: 161173

ΝΕΡΟΥΤΣΟΣ ΝΕΚΤΑΡΙΟΣ - ΑΜ: 171090

ΧΑΤΖΟΠΟΥΛΟΣ ΗΛΙΑΣ - ΑΜ: 47434

Εισαγωγή.....	3
Ακέραιοι Αριθμοί	3
Αριθμοί Κινητής Υποδιαστολής.....	3
Συμβολοσειρές.....	3
Αναγνωριστικά	3
Διάγραμμα Μετάβασης.....	4
Πίνακες Μεταβιβάσεων	5
Regular Expression.....	6
Κώδικας Uniform FSM.....	7
Ενδεικτικά Τρεξίματα.....	10
• Identifiers.....	10
• Integers	10
• Strings	11
• Floating Point.....	12

Εισαγωγή

Σε αυτό το εργαστηριακό μέρος σκοπός μας ήταν η υλοποίηση και η κωδικοποίηση ενός αυτόματου το οποίο βασίζεται στο πρότυπο της γλώσσας προγ/σμού uni-C. Το συγκεκριμένο αυτόματο είναι ικανό να αναγνωρίζει integer tokens (dec, octet, hex), float tokens και identifier tokens.

Για την υλοποίηση του αυτόματου δημιουργήσαμε τον κατάλληλο πίνακα μετάβασης και τις κανονικές εκφράσεις. Η κωδικοποίηση του αυτόματου έγινε με την χρήση πεπερασμένων καταστάσεων και την εκτέλεσή τους στο εργαλείο FSM.

Η διάκριση των tokens κρίνεται από τον πρώτο χαρακτήρα που διαβάζεται από τον χρήστη ως εξής:

Ακέραιοι Αριθμοί

- **Δεκαδικοί:** Ο πρώτος χαρακτήρας είναι '1' έως '9', και μετά σε συνέχεια οποιοδήποτε πλήθος αριθμών από '0' έως '9' [π.χ 1000, 500, 1234, 99000.....0].
- **Οκταδικοί:** Ο πρώτος χαρακτήρας είναι '0' ακολουθούμενος από ένα οποιοδήποτε πλήθος αριθμών από '0' έως '7' [π.χ 010, 0172, 063421].
- **Δεκαεξαδικοί:** Ο πρώτος χαρακτήρας είναι '0', ο δεύτερος 'x X' και μετά οποιοδήποτε πλήθος χαρακτήρων οι οποίοι μπορούν να είναι από '0' έως '9' ή 'a A' έως 'f F' [π.χ 0xAEF...].

Αριθμοί Κινητής Υποδιαστολής

Οι Αριθμοί Κινητής Υποδιαστολής μπορούν να ξεκινάνε με αριθμούς από '0' ή '1' έως '9', στην περίπτωση όπου ξεκινάνε με '0' πρέπει να έχουν δεύτερο χαρακτήρα '.' ή 'e E' ενώ στην περίπτωση '1' έως '9' μπορεί να ακολουθείται από οποιοδήποτε πλήθος αριθμών από '0' έως '9', σε συνέχεια πρέπει να υπάρχει από αυτήν την ακολουθία ο χαρακτήρας '.' ή 'e E'.

Εφόσον εισαχθεί ο χαρακτήρας '.' πρέπει να εισαχθεί τουλάχιστον ένας αριθμός από '0' έως '9', αφετέρου μπορεί να εισαχθεί οποιοδήποτε πλήθος αριθμών στο ίδιο όριο, προαιρετικά στο τέλος της ακολουθίας μπορεί να εισαχθεί ο χαρακτήρας 'e E'.

Μετά τον χαρακτήρα 'e E' μπορεί προαιρετικά να ακολουθεί σύμβολο προσήμου '-' και σε συνέχεια πρέπει να εισαχθεί είτε ο αριθμός '0' είτε αριθμός από το 1 έως 9, αν εισαχθεί '0' ολοκληρώνεται ο αριθμός, αν εισαχθεί '1' έως '9' μπορεί να ακολουθήσει οποιοδήποτε πλήθος αριθμών από '0' έως '9'.

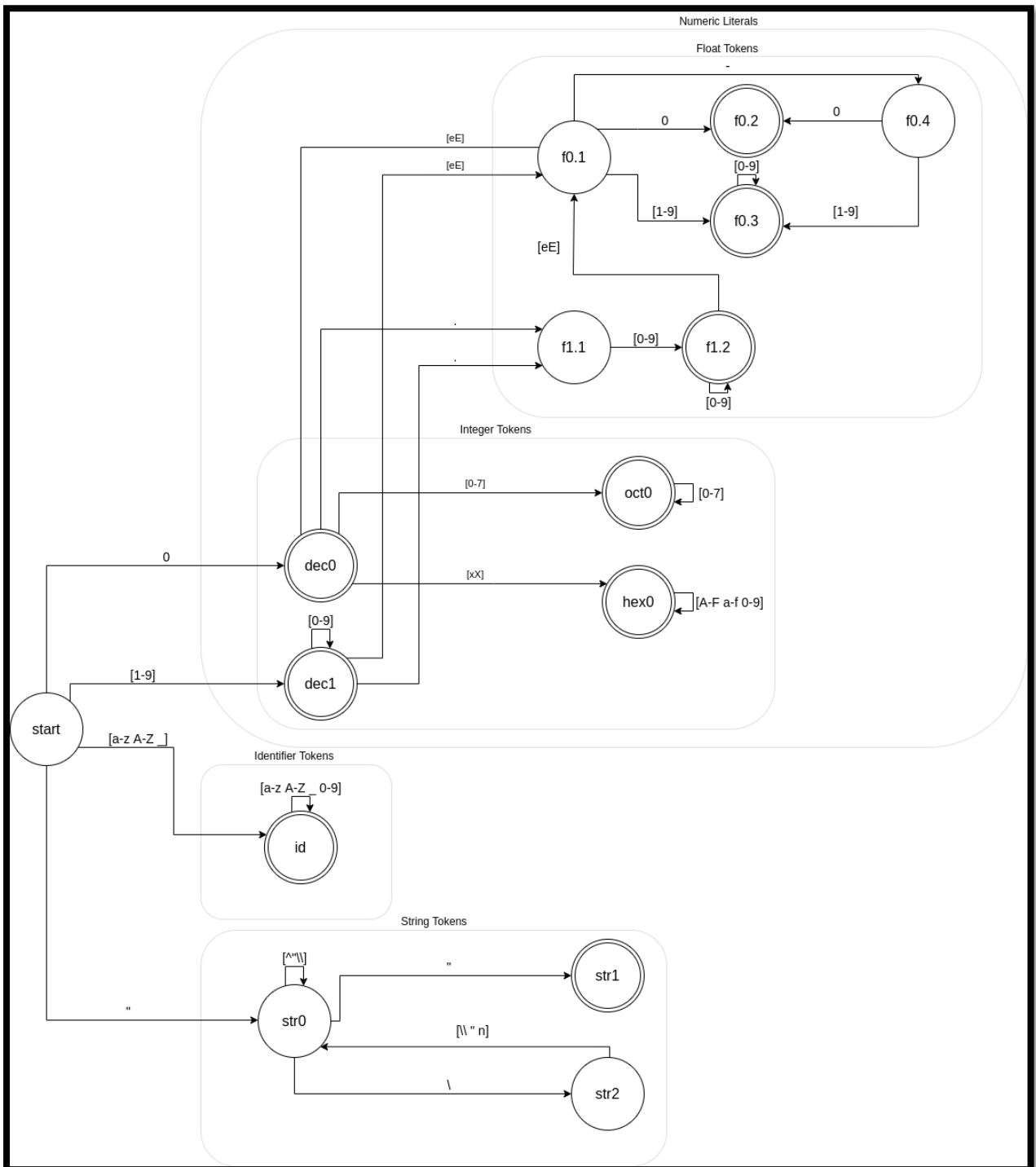
Συμβολοσειρές

Οι Συμβολοσειρές αναγνωρίζονται όταν ο πρώτος χαρακτήρας είναι '“' όπου πρέπει να τελειώνουν με τον ίδιο χαρακτήρα. Μέσα στις συμβολοσειρές μπορεί να ακολουθήσει οποιαδήποτε σειρά από χαρακτήρες μέχρι να εμφανιστεί δεύτερη φορά ο χαρακτήρας '“'. Ειδική περίπτωση είναι οι χαρακτήρες '“' και ' / ', για να μπορέσουν να εισαχθούν στην σειρά πρέπει να προηγείται ο χαρακτήρας ' / ' [π.χ /” , //].

Αναγνωριστικά

Τα αναγνωριστικά αναγνωρίζονται όταν οι πρώτοι χαρακτήρες είναι '_' ή 'A' έως 'Z'. Οι επόμενοι χαρακτήρες που ακολουθούν μπορούν να έχουν επίσης και τους αριθμούς '0' έως '9' [π.χ _example01].

Διάγραμμα Μετάβασης



Πίνακες Μεταβιβάσεων

Καταστάσεις	Uniform																									
	Χαρακτήρες εισόδου																									
	0	1-7	8	9	A-D	E	F	G-W	X	Y	Z	a-d	e	f-m	n	o-w	x	y	z	_	.	-	"	\	ln	*
start	dec0	dec1	dec1	dec1	id	id	id	id	id	id	id	id	id	id	id	id	id	id	id	id	bad	bad	str0	bad	bad	bad
dec0	oct0	oct0	bad	bad	bad	f0.1	bad	bad	hex0	bad	bad	bad	f0.1	bad	bad	bad	hex0	bad	bad	bad	f1.1	bad	bad	bad	good	bad
dec1	dec1	dec1	dec1	dec1	bad	f0.1	bad	bad	bad	bad	bad	bad	f0.1	bad	bad	bad	bad	bad	bad	bad	f1.1	bad	bad	bad	good	bad
id	id	id	id	id	id	id	id	id	id	id	id	id	id	id	id	id	id	id	id	id	bad	bad	bad	bad	good	bad
str0	str0	str0	str0	str0	str0	str0	str0	str0	str0	str0	str0	str0	str0	str0	str0	str0	str0	str0	str0	str0	str0	str0	str1	str2	bad	str0
str1	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	good	bad
str2	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	str0	bad	bad	bad	bad	bad	bad	bad	str0	str0	bad	bad
oct0	oct0	oct0	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	good	bad
hex0	hex0	hex0	hex0	hex0	hex0	hex0	hex0	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	good	bad
f0.1	f0.2	f0.3	f0.3	f0.3	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad
f0.2	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	good	bad
f0.3	f0.3	f0.3	f0.3	f0.3	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	good	bad
f1.1	f1.2	f1.2	f1.2	f1.2	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad
f1.2	f1.2	f1.2	f1.2	f1.2	bad	f1.3	bad	bad	bad	bad	bad	bad	f1.3	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	good	bad
f1.3	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad
f1.4	bad	f1.5	f1.5	f1.5	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	good	bad
f1.5	f1.5	f1.5	f1.5	f1.5	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	bad	good	bad

Regular Expression

Identifiers:	Regex:	/[A-z_]+[A-z_\d]*/g
	EBNF:	Identifier = Id_Start, Id_Tail; Id_Start = {Letter “_”}-; Id_Tail = (Letter Digit “_”)*; Letter = ? A...z ?; Digit = ? 0...9 ?;
Strings:	Regex:	/"^[^\\\"\\n]*(\\\\\\ \\\\\\\" \\\\\\n)*"/g
	EBNF:	String = String_Bracket, {String_Contents}, String_Bracket; String_Bracket = “”; String_Contents = (Letter Digit “\\” ‘\\’ “\n” “String_Special_Chacters”)*; String_Special_Chacters = ? Special Characters Without \ “ \n ?;
Integers:	Regex:	/0[xX][\dA-f]+ 0[0-7]* [1-9]\d*/g
	EBNF:	Ints = Hex Oct Int ; Hex = “0”, Hex_Identifier, Digit Hex_Numbers, {Digit Hex_Numbers}; Oct = “0”, Oct_Numbers, {Oct_Numbers}; Int = {Non_Zero_Digit}-, {Digit}; Hex_Identifier = “x” “X”; Hex_Numbers = ? A...f ?; Oct_Numbers = ? 0...7 ?; Digit = ? 0...9 ?; Non_Zero_Digit = ? 1...9 ?;
Floats:	Regex:	/([0-9][1-9][0-9]*)+((\.[0-9]+)([eE][-]?([0-9][1-9][0-9]*)?) ([eE][-]?([0-9][1-9][0-9]*))) /g
	EBNF:	Float = (“0”, (“.”, Int_Rule) (Notation, Notation_Rule)) ({Non_Zero_Digit}-, (“.”, Int_Rule) (Notation, Notation_Rule)); Notation = “E” “e”; Int_Rule = {Digit}-, [Notation, Notation_Rule]; Notation_Rule = [“-”], (“0” Non_Zero_Digit, {Digit}); Digit = ? 0...9 ?; Non_Zero_Digit = ? 1...9 ?;

Κώδικας Uniform FSM

```
start: 0          -> dec0
      1-9         -> dec1
      a-z A-Z _    -> id
      "           -> str0
      *           -> BAD
```

// NUMERIC LITERALS

```
dec0: E e         -> f0.1
      .           -> f1.1
      0-7         -> oct0
      x X         -> hex0
      \n          -> GOOD
      *           -> BAD
```

```
dec1: 0-9         -> dec1
      .           -> f1.1
      e E         -> f0.1
      \n          -> GOOD
      *           -> BAD
```

// IDENTIFIER TOKENS

```
id: a-z A-Z _ 0-9 -> id
   \n          -> GOOD
   *           -> BAD
```

// STRING TOKENS

```
str0: "          -> str1
      \\          -> str2
      \n          -> BAD
      *          -> str0
```

```
str1: \n          -> GOOD
      *          -> BAD
```

```
str2: \\          -> str0
      "          -> str0
      n          -> str0
      *          -> BAD
```

// INTEGER TOKENS

```
oct0: 0-7         -> oct0
      \n          -> GOOD
      *          -> BAD
```

```
hex0: A-F 0-9     -> hex0
\n              -> GOOD
*              -> BAD
```

// FLOAT TOKENS

f0.1: 0	-> f0.2
1-9	-> f0.3
-	-> f0.4
*	-> BAD

f0.2: \n	-> GOOD
*	-> BAD

f0.3: 0-9	-> f0.3
\n	-> GOOD
*	-> BAD

f0.4: 0	-> f0.2
1-9	-> f0.3

f1.1: 0-9	-> f1.2
*	-> BAD

f1.2: 0-9	-> f1.2
e E	-> f0.1
\n	-> GOOD
*	-> BAD

GOOD(OK): \n	-> GOOD
--------------	---------

- Strings

```
C:\Users\eprep\Desktop\My Files>fsm uniform.fsm
"Hello world"
YES
^C
C:\Users\eprep\Desktop\My Files>fsm uniform.fsm
"!^<> :D hello!"
YES

C:\Users\eprep\Desktop\My Files>fsm uniform.fsm
""
fsm: in uniform.fsm, state 'bad' input \n not accepted

C:\Users\eprep\Desktop\My Files>fsm uniform.fsm
"\
NO

C:\Users\eprep\Desktop\My Files>fsm uniform.fsm
"Mixed String""
fsm: in uniform.fsm, state 'bad' input " not accepted

C:\Users\eprep\Desktop\My Files>fsm uniform.fsm
"\
YES
^C
C:\Users\eprep\Desktop\My Files>fsm uniform.fsm
"\n"
YES

C:\Users\eprep\Desktop\My Files>fsm uniform.fsm
"\"
YES
```

- Floating Point

```
C:\Users\eprep\Desktop\My Files>fsm uniform.fsm
10.0
YES

C:\Users\eprep\Desktop\My Files>fsm uniform.fsm
0.10.10
fsm: in uniform.fsm, state 'bad' input 1 not accepted

C:\Users\eprep\Desktop\My Files>fsm uniform.fsm
10e10
YES
^C

C:\Users\eprep\Desktop\My Files>fsm uniform.fsm
0e10
YES
^C

C:\Users\eprep\Desktop\My Files>fsm uniform.fsm
0e0
YES

C:\Users\eprep\Desktop\My Files>fsm uniform.fsm
10e10e10e10
fsm: in uniform.fsm, state 'bad' input 1 not accepted

C:\Users\eprep\Desktop\My Files>fsm uniform.fsm
10e-10
YES
^C

C:\Users\eprep\Desktop\My Files>fsm uniform.fsm
10.5000e-10
YES
^C

C:\Users\eprep\Desktop\My Files>fsm uniform.fsm
10.50e10.1000
fsm: in uniform.fsm, state 'bad' input 1 not accepted
```